KV-cache capacity is not serving capacity

Working title. In a continuously-batched LLM server, the signal used to govern capacity is non-monotonically related to the failure it is used to prevent, and which resource binds is itself a scheduler setting.

Status of this draft

This is a **manuscript draft assembled from the analysis documents in this repository**, not a new analysis. Every number below is transcribed from a generated artefact and cited to it:

Source	What it supplies
`docs/queueing_model.md`	The service-rate model, its fitted parameters, and §5's limitations
`docs/experiment_b.md`	Experiments A, B and F — the memory boundary and the scorer law
`results/stats/STATISTICS.md`	Phase-2 policy comparison and Experiment D
`results/expF/expF_analysis.json`	Experiment F's per-arm agreement
`snapshots/paper-v2/ENVIRONMENT.json`	Environment capture at freeze time

Two conventions, because they are the difference between a defensible paper and a retracted one:

- **Measured, asserted, and derived are labelled separately.** §10 lists every claim by status.
- **Numbers are not restated from memory.** Where a figure appears here it exists in a generated file; where a figure does *not* exist, this draft says so rather than estimating it.

Citation status (applies to §2 and §15). No reference is written from memory, and every quotation was transcribed from the paper itself rather than from a search result or a fetch summary — a summariser misreported one of these papers' central numbers during drafting, which is why the rule exists. Eight of the ten references were obtained in full and the cited passages read directly (**[R]** in §15); the three not obtained (**[M]** — the two paywalled LPS papers and FastServe) are cited only for claims an [R] source corroborates or for a mechanism named in the title. **No `[CITE]` gaps remain**; §14 records how the last five were closed, including two that were closed by labelling a claim honestly rather than by finding a source for it.

Format. Markdown, because this machine has no LaTeX or pandoc toolchain (`pdflatex`, `xelatex`, `latexmk`, `tectonic`, `pandoc` all absent) and the rest of `docs/` is Markdown. Converting to a venue template is mechanical once a venue is chosen; no venue or format was specified, so this draft is venue-neutral. See §14 for the open items that block submission.

Abstract

Submission length, ~215 words. The extended version below carries the same claims with the evidence attached; keep the two in step.

On a single Tesla T4 running vLLM 0.8.5.post1 with Qwen2.5-1.5B and 3B under n-gram speculative decoding, KV-cache occupancy — the signal serving systems use to govern admission and capacity — is **non-monotonically related to failure**. Across ten trials of one configuration, every survivor peaked at a higher occupancy than every death (survived at 82.6%, died at 57.4%, no overlap), so no threshold on the signal separates them. The binding resource lies outside the paged pool: in 16 engine deaths, every failure was a dense `[N, k+1, V]` fp32 tensor in the speculative-decoding scorer, invisible to every admission signal the engine exposes. Its size follows `M = N·(k+1)·V·4` bytes, predicting the failing allocation to within 0.1% at k = 4 and k = 7 and 0.5% at N ≈ 410; the `V` term is read from the source rather than measured. Disabling speculative decoding removes the failure (0/3 versus 3/3) at a higher occupancy than killed the speculative arm. Which resource binds is itself a scheduler setting: raising `max_num_seqs` from 256 to 1024 shifts it to the KV pool and halves sustainable concurrency, because the cap sizes an activation reserve taken from that pool. The failing path is absent from vLLM's V1 engine, and we claim no generality beyond this system.

Extended summary

Continuously-batched LLM serving systems are provisioned as though the paged KV cache were the binding resource: admission control, capacity planning and autoscaling all key on KV occupancy. We test that assumption on one deployment — a single Tesla T4 running vLLM 0.8.5.post1 with Qwen2.5-1.5B and 3B under n-gram speculative decoding — and find the signal not merely imprecise but **non-monotonically related to failure**. Across ten repeated trials of one configuration, every survivor peaked at a *higher* KV occupancy than every death: the engine survived a full load ramp at 82.6% having died at 57.4%, with a 20-point gap and no overlap between the two groups. No threshold on occupancy separates them, because the resource that binds is not in the pool the signal measures.

Which resource binds turns out to be a scheduler setting. Raising `max_num_seqs` from 256 to 1024 — nominally an admission limit — moves the binding constraint from an unmonitored allocation to the KV pool on the same model, GPU and workload, because the raised cap enlarges vLLM's activation reservation and shrinks the pool the same setting is presumed to govern, halving the concurrency the engine can sustain.

We then identify what binds. In 16 engine deaths across two model sizes, three experiments and two speculative depths, every failure occurred in one allocation: a dense `[N, k+1, V]` fp32 tensor in the speculative-decoding scorer, outside the paged allocator and invisible to every admission signal the engine exposes. That speculative decoding costs memory at high concurrency is known; what we add is an exact form and a test of it. `M = N·(k+1)·V·4` bytes predicts the failing allocation to within 0.1% at k = 4 and k = 7 and to within 0.5% at N ≈ 410, and correctly predicts that k = 2 does not fail at all at the default sequence cap. **The `V` term is read from the allocation site rather than measured** — every death in the campaign ran a checkpoint with V = 151,936 — so the

law is tested on two of its three axes. A matched control closes the causal argument: disabling speculative decoding removes the failure entirely (0/3 versus 3/3) at a *higher* occupancy than the one that killed the speculative arm.

Finally, in one workload cell at n = 10, admission control governs **aggregate throughput as well as service differentiation** (+178 tok/s and +0.888 SLO attainment for an uncapped queue over FCFS, both surviving Holm correction across a 207-comparison family), where the common framing has admission policy trading latency for fairness at fixed capacity.

Scope. These are measurements of one machine, one engine version and one model family, on a passively cooled device that thermally throttled through nearly the whole campaign — so absolute rates are depressed and only paired comparisons should be read as hardware-independent. The specific failing code path does not exist in vLLM's V1 engine, which scores on a flattened tensor instead. We therefore do not claim that these constants generalise, nor that this particular allocation binds elsewhere. The claim is narrower: in this system the signal used to govern capacity was non-monotonically related to the failure it is used to prevent, and the regime boundary was movable by a parameter that does not announce itself as a memory parameter. Whether other engines also admit against resources they cannot see is an open empirical question that this paper motivates rather than settles.

---

1. Introduction

A continuously-batched LLM server admits requests into a running batch, advances every resident sequence by one token per engine step, and evicts sequences as they finish. The dominant memory consumer in this design is the paged KV cache, and the dominant operational assumption is that KV capacity *is* serving capacity: if the pool has room, the system can take the work.

That assumption is load-bearing. It is why vLLM exposes KV occupancy as a first-class gauge, why admission controllers throttle on it, and why capacity planning reduces to "how many sequences fit in the pool". The finding that organises this paper is a pair of runs that breaks it without needing any model at all:

> Two trials of one configuration — same engine, same workload, same seed, same machine, eight minutes apart. The first **died with the KV pool 57.4% full**. The third **survived the entire ramp at 80.7%** and kept serving. Across the full set of ten trials the widest such pair is a death at 57.4% against a survival at **82.6%**.

Occupancy is therefore not merely a weak predictor of failure in this system. It is not monotonically related to it, so no threshold on the signal can separate the two runs. Something outside the pool is binding, and the gauge that operators watch cannot see it.

The rest of the paper answers two questions that follow. **What is binding?** (§7) — an allocation in the speculative-decoding scorer, outside the paged allocator, whose size we give in closed form and test. **When does it bind?** (§8) — a question with an uncomfortable answer, since the regime boundary turns out to be movable by a scheduler parameter that does not look like a memory parameter.

1.1 Contributions

1. **Occupancy is non-monotonically related to failure** (§7.1). Deaths occurred with 37–56% of the KV pool free, and matched trials survived at 82.6% having died at 57.4%. This is the paper's central result and it requires no model: two runs of one configuration, one dead at low occupancy and one alive at high.
2. **The binding resource is selectable by a scheduler parameter** (§8). Changing `max_num_seqs` from 256 to 1024 moves the binding constraint from the scorer to the KV pool on the same model, GPU and workload, because the raised cap costs 3.1 GiB of activation reservation taken out of the KV pool. Raising the admission limit **halves** the concurrency the engine can sustain.
3. **A matched mechanism control** (§7.5). With speculative decoding disabled the same ramp survives 3/3 at 99.91–99.96% occupancy, higher than the 98.66–98.96% at which the speculative arm died 3/3. Removing the allocation removes the failure, at an occupancy that would have triggered it were occupancy the constraint.
4. **A closed-form law for the allocation, tested on two axes** (§7.2–7.4). `N·(k+1)·V·4` bytes predicts the failing allocation to within 0.1% across N from 220 to 410 and $k \in \{2, 4, 7\}$. The V term is read from the source, not measured, and we say so.
5. **Admission control governs aggregate capacity** (§9). At n = 10, an uncapped queue beats FCFS by +178 tok/s *and* +0.888 SLO attainment, both surviving Holm correction over 207 comparisons — where the n = 3 campaign had yielded a single surviving contrast in 171, and not a capacity one.
6. **A service-rate model and an honest account of where it fails** (§5, §6). $\mu(N) = \min(N \cdot r_0, \mu\_max)$ fits one workload at $R^2 = 0.983$ and the pooled data at $R^2 \approx 0.22$. We report the second number as prominently as the first, because the model's failure to generalise is itself the finding that motivates §7.

1.2 Relation to what is already known

Two of the ingredients here are not new, and the paper is weaker if it pretends otherwise.

That speculative decoding costs memory at high concurrency is known, as is the fact that it degrades at large batch sizes, where the target model is already compute-efficient and the extra `k` verification positions are overhead. The existing treatment of this is a *speedup* question, measured in throughput. Our contributions on this axis are narrower and different in kind: an exact closed form rather than a scaling statement, a test of that form on the `k` axis, and the observation that the allocation is not a tax on throughput but the thing that **ends the engine's life** — a survival question, which is what makes it a capacity-planning problem rather than a tuning one.

Limited processor sharing is established queueing theory (§4). We adopt it as the correct description of a batch-parallel server; we do not claim it. What is not in that literature is the coupling in §8, where the multiprogramming limit and the memory constraint are entangled by the implementation, so that raising the limit shrinks the resource the limit governs.

The novel core is §7.1 and §8: a capacity signal that is non-monotonically related to the failure it is used to prevent, and a regime boundary movable by a parameter that does not announce itself as a memory parameter.

1.3 What this paper does not claim

The failure we characterise is specific to vLLM's V0 engine (§10.2), to a single T4 (§10.1) and to a model family

with one vocabulary size (§10.3). We do not claim speculative decoding is a bad design — it is a net win in the regime it was built for — nor that we have found a defect: the allocation does exactly what it was written to do. The finding is that its size is unbounded in a dimension nothing admits against. The *mechanism* — an allocation scaling with `N × (k+1) × V`, outside the paged allocator and invisible to admission control — is a design property whose persistence in other engines is an open empirical question that nothing here settles.

---

2. Background and related work

2.1 Continuous batching and paged memory

Orca [1] introduced **iteration-level scheduling**, in which the scheduler "invokes the execution engine to run only a single iteration of the model on the batch" [1, §1], so requests join and leave between iterations. `N`, the number of resident sequences — vLLM's `num_requests_running` gauge — therefore varies continuously within a run, bounded by `max_num_seqs` (default 256).

Orca also introduced the admission-time memory discipline that this paper's failures evade, and it is worth stating precisely because it is sound. Its scheduler takes two operator knobs: `max_bs`, the largest number of requests in a batch, and `n_slots`, "the size of memory region (in terms of slots) allocated to the Attention K/V manager" [1, §4.2]. Since KV buffers cannot be reclaimed until a request finishes, a naive scheduler can deadlock; Orca reserves `max_tokens` slots at admission and admits only if they fit, giving a guarantee:

> "if the reservation is possible, it is guaranteed that the manager can allocate buffers for the newly generated keys and values until the request finishes." [1, §4.2]

That guarantee is exactly as wide as the allocator it ranges over. An allocation made elsewhere is neither reserved against at admission nor visible to the accounting that makes it true — and §7 reports failures of that kind, with the reservation discipline intact throughout.

vLLM [2] added **PagedAttention**, storing the KV cache in non-contiguous fixed-size blocks "inspired by the operating system's solution to memory fragmentation and sharing: virtual memory with paging" [2, §1], and with it the occupancy gauge operators watch. The premise that makes occupancy a capacity signal is stated explicitly there. Kwon et al. give the layout for a 13B model on a 40 GB A100 as 65% parameters, close to 30% KV cache, and a small remainder for activations, and conclude:

> "Since the model weights are constant and the activations only occupy a small fraction of the GPU memory, the way the KV cache is managed is critical in determining the maximum batch size." [2, §1]

Both halves of that reasoning fail on the system we measure. The activation reservation is not a fixed small fraction: it scales with `max_num_seqs`, from 2.52 GiB at a cap of 256 to 5.64 GiB at 1024, taken out of the KV pool (§8). And what determines the maximum batch size is neither the weights nor the KV cache but a third allocation, in the speculative-decoding scorer, that the block manager does not account for and no gauge reports (§7).

2.2 Speculative decoding and its costs

Speculative decoding was introduced concurrently and independently by Leviathan et al. [3] and Chen et al. [4]: a cheap draft proposes γ tokens, the target scores them in parallel, and modified rejection sampling accepts a prefix while leaving the target's output distribution unchanged. Our runs use vLLM's prompt-lookup n-gram drafter, so no draft model's weights enter the memory budget — which matters because the allocation in §7 cannot be attributed to one.

What both papers account for is bandwidth and arithmetic, not memory capacity, and that is the gap this paper falls into. The method's stated justification is that decoding is bandwidth-bound and therefore has compute to spare:

> "inference from large models is often not bottlenecked on arithmetic operations, but rather on memory bandwidth and communication, so additional computation resources might be available." [3, §1]

Chen et al. reason identically, and where they discuss the KV cache growing with batch size it is again a *bandwidth* concern [4, Related Work]. Neither analyses the memory *capacity* consumed by materialising scores for `k+1` positions across a batch. The trade is compute-for-latency and on that accounting it is favourable; the resource we find binding does not appear in it. Where the scaling is stated elsewhere it is qualitative — the scorer is "K× more expensive" — whereas §7.3–7.4 give an exact expression and test it on two of its three axes.

The batch-size regime matters for the same reason: the premise is that spare compute exists, and at high concurrency it may not. Sadhukhan et al. [11] record the resulting consensus — "the conventional wisdom suggests that [speculative decoding's] efficacy is limited to small batch sizes" [11, abstract] — and attribute it to verification cost, since "with large batches, LLMs become compute bound, making verification significantly costly", so that "the usage of SD in high batch size regime is discouraged by existing works" [11, §1]. They then qualify it: past a critical sequence length KV loading dominates again and speculation recovers, which is the regime their own method targets. Our runs sit below that length, where the discouragement applies — but the cost we measure is not the verification *compute* that this literature weighs. It is the memory the verification step allocates, which neither the original papers nor their large-batch critics account for. Implementation is not incidental either: we measure an engine that expands the batch into a padded `[N, k+1, V]` tensor, while vLLM's V1 engine scores a flattened one (§10.2).

2.3 Admission control and SLO-aware scheduling

A substantial line of work schedules LLM requests against latency and SLO targets: Sarathi-Serve [5] overlaps chunked prefills with ongoing decodes; Llumnix [6] live-migrates requests with their GPU memory state across instances; QLM [7] reorders a queue against estimated waiting times; FastServe [12] preempts at the granularity of a single output token, using a skip-join multi-level feedback queue to attack head-of-line blocking.

These systems do change aggregate throughput — QLM reports 20–400%, Sarathi-Serve 2.6–3.7× serving capacity — **so §9's result is not novel in that respect. What differs is the mechanism.** Their instruments are utilisation and multiplexing, and memory enters each design as KV space: Llumnix's motivating problem is that "varying request lengths and memory demands inevitably result in memory fragmentation across instances" [6, §1], and Agrawal et al. describe Orca and vLLM as eagerly scheduling prefills "whenever GPU memory becomes

available" [5, §1] — admission keyed on exactly the resource §7 shows is not binding, inheriting Orca's guarantee along with its scope.

We read §9's gain — an uncapped queue adding +178 tok/s over FCFS in the cell where it adds +0.888 SLO attainment — as a consequence of §7 rather than a scheduling insight: a policy holding concurrency below the unmonitored boundary keeps the engine alive, and a live engine outproduces a dead one. The contribution is not that admission affects throughput, but that near such a boundary it does so through a channel none of these signals expose.

2.4 Queueing models for batch-parallel servers

§4 argues that a continuously-batched engine is a **limited processor sharing** (LPS) server rather than a processor-sharing one. In LPS "the server can serve up to $K \geq 1$ jobs simultaneously, equally distributing its attention to each of them", and "letting $K = \infty$ makes the system a standard processor sharing (PS) queue" [10, §1]. This is established theory and we claim no part of it: Zhang and Zwart [8] give steady-state heavy-traffic approximations; Zhang, Dai and Zwart a fluid approximation [9] and a diffusion limit [10]. Our use is descriptive, and matters because the wrong label (M/G/1-PS) implies a fixed aggregate capacity this system lacks at low concurrency.

Where our system departs is in what setting the limit costs. That limit is ordinarily independent of the memory constraint: Orca exposes `max_bs` and `n_slots` as separate operator knobs [1, §4.2], and the LPS literature justifies K by scheduling overhead — "allowing too many jobs to time-share at once can lead to significant overhead due to switching" [10, §1] — so K is exogenous, entering the analyses only through the scaling.

In the engine we measure, `max_num_seqs` plays K's role and is *not* free in this sense: it both bounds concurrency and sizes the activation reservation deducted from the KV pool, so raising it lowers the residency it is meant to raise (§8). At $K = 1024$ the pool falls to 1.63 GiB and the engine sustains $N \approx 125$, against $N = 256$ at $K = 256$. Orca's two knobs have been collapsed into one, and the sign of the effect inverts over part of the range. We have not found this coupling treated in the LPS literature, though our reading is limited to the three papers cited.

---

3. Experimental setup

Single-node, single-GPU. **Figure 1** shows the harness: an admission layer in front of an unmodified vLLM engine, which is what lets §9 vary admission policy without touching the engine whose failure §7 characterises. Full capture in `snapshots/paper-v2/ENVIRONMENT.json`.

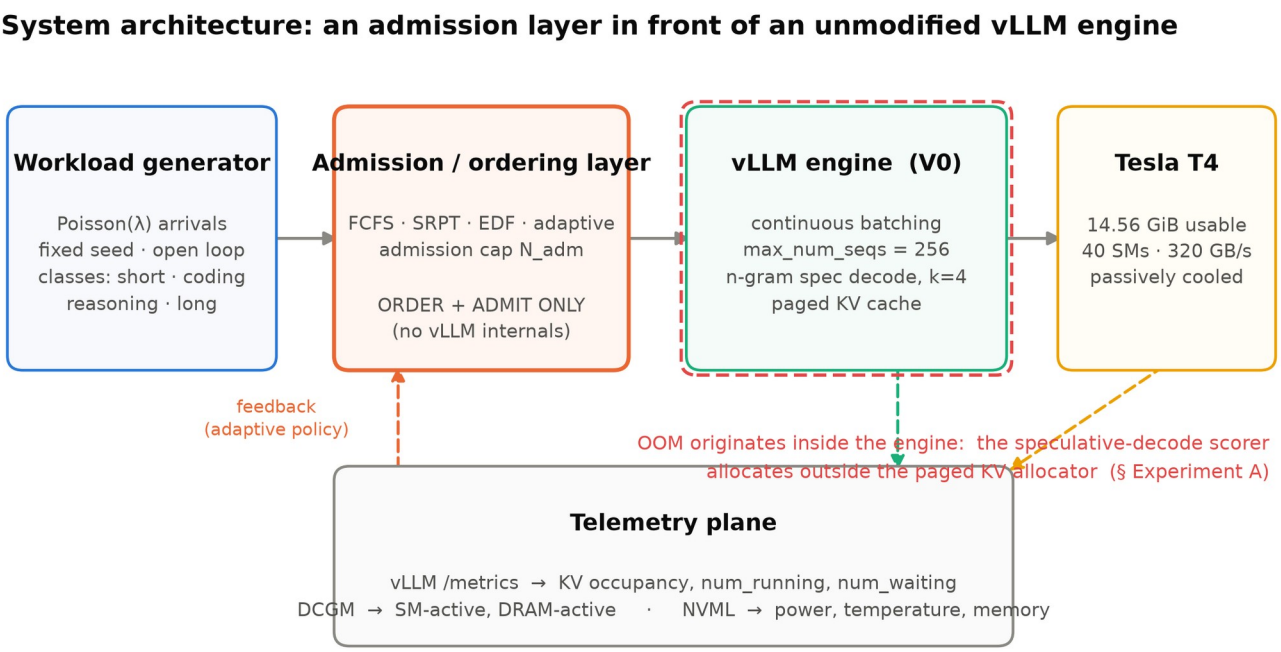

Figure 1. The harness: an admission layer in front of an unmodified vLLM engine. This is what lets §9 vary admission policy without touching the engine whose failure §7 characterises.

GPU	Tesla T4, 14.56 GiB usable, CC 7.5, 40 SMs, 70 W cap
Host	Intel Xeon E5-2696 v4, 88 threads, Linux 6.8.0-124-generic
Driver	580.173.02 (kernel module and userspace NVML agree)
Stack	vLLM 0.8.5.post1, PyTorch 2.6.0+cu124, CUDA 12.4, Python 3.11.15
Models	Qwen2.5-1.5B-Instruct, Qwen2.5-3B-Instruct, float16
Speculation	n-gram prompt-lookup, `k` = 2/4/7, `prompt_lookup` 2–5

Arrivals are Poisson with a fixed seed, open-loop (`common/loadgen.py`); service requirement is the output length, drawn per workload class. Telemetry is sampled per stage: `num_running`, `num_waiting`, KV occupancy, memory, temperature, power and the NVML throttle flag.

The device throttles. Nearly every run in the campaign sat at 85–89 °C with the NVML thermal-throttle flag set. This is a passively cooled T4 in a dense chassis. Every absolute rate in this paper is therefore depressed relative to an unthrottled device, and **only paired and ratio comparisons should be read as hardware-independent**. This is stated once here and again in §10.4 because it conditions every throughput number in §5.

---

4. Why "M/G/1-PS" is the wrong label

Three of the four Kendall symbols hold: arrivals are Poisson by construction, service requirements are generally

distributed by construction, and there is one server. **Processor sharing does not hold.** Classical M/G/1-PS assumes a fixed capacity μ shared among N jobs, each receiving μ/N. A continuously-batched engine does not behave that way at low concurrency: every step advances all N sequences, and at small N the step time is dominated by streaming weights, a cost paid once per step regardless of N. Adding a sequence therefore costs almost nothing and the **aggregate rate grows with N**.

The correct description is an M/G/1 queue with a concurrency-dependent service rate, batch- parallel below a knee and processor-sharing above it, subject to a hard multiprogramming limit — **limited processor sharing**, not PS.

We adopt LPS; we do not claim it. It is established queueing theory with a substantial literature on fluid and diffusion limits and heavy-traffic approximations, and the contribution here is only the observation that a continuously-batched LLM engine is an instance of it. What is *not* in that literature is the coupling §8 reports: the multiprogramming limit is not a modelling convenience but `max_num_seqs`, and raising it **shrinks the memory the limit is supposed to govern**. In LPS theory the multiprogramming limit is a free parameter; here it is entangled with the resource constraint by the implementation, and the two cannot be set independently.

---

5. A concurrency-dependent service-rate model

$$\mu(N) = \min(N \cdot r_0, \mu\_max) \qquad N^* = \mu\_max / r_0 \qquad 0 \le N \le N\_max$$

r_0 is the per-sequence generation rate in the uncontended regime; `μ_max` is the saturated aggregate rate. Both are **empirically fitted**, neither derived from hardware specifications. Fitting is non-linear least squares over run-level `(N, X)` pairs, with 95% CIs from a non-parametric bootstrap **over runs** (2000 resamples), so the run-to-run variance structure — the dominant noise source — is preserved.

From `docs/queueing_model.md` §4:

Workload	n	N range	r_0 (tok/s/seq)	μ_max (tok/s)	N*	R^2
`phase1_mixed`	9	1.4 – 60.6	**12.60** [11.93, 14.86]	**247.5** [240.3, 253.0]	**19.7** [16.7, 20.8]	**0.983**
`expB_qwen3b`	18	0.9 – 129.2	4.06 [3.85, 4.59]	*470.8*	*116.1*	0.950
`phase2_mixed`	19	3.6 – 18.2	50.57 [45.46, 54.35]	*451.9*	*8.9*	0.862
`modelval_mixed`	17	0.4 – 116.8	8.65 [6.61, 12.31]	*355.7*	*41.1*	0.825
`expB_qwen1.5b`	17	0.4 – 117.0	8.52 [6.36, 12.21]	*355.2*	*41.7*	0.813
`phase2_chat`	23	3.6 – 11.8	58.23 [56.05, 61.01]	*349.2*	*6.0*	0.479
`phase1_validation`	6	1.2 – 32.6	63.27	*141.2*	*2.2*	0.372
pooled	109	0.4 – 129.2	—	—	6.9	**0.229**

Italicised μ_max and N are extrapolations, not measurements (§6.1).* Only `phase1_mixed` both spans the knee

generously and has a flat plateau (slope −0.61 tok/s/seq, indistinguishable from zero at this noise level; MAPE 12.7%). It is the reference fit, and **Figure 8** shows it against its measurements; **Figure 2** shows the throughput curves for all workloads, where the workload-specificity of the knee is visible directly.

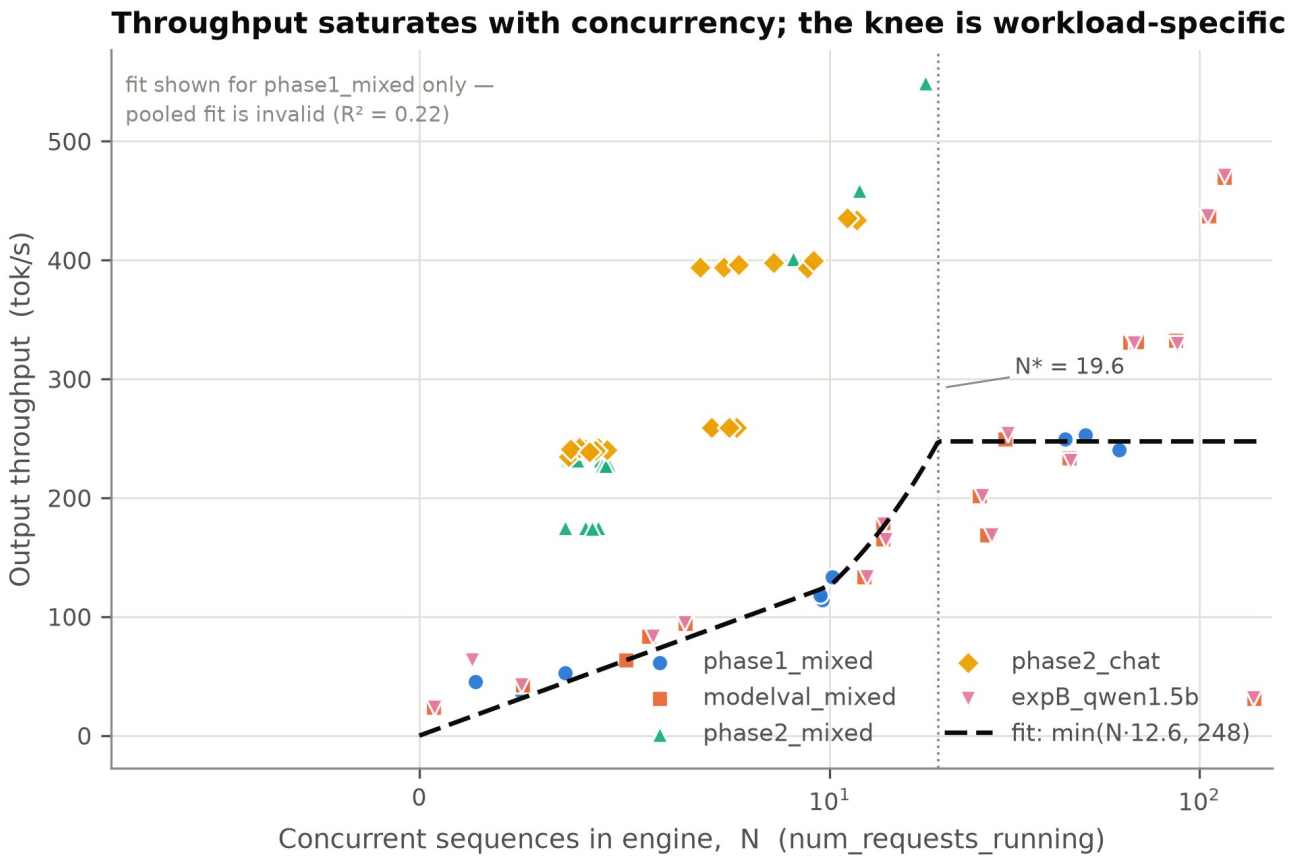

Figure 2. Throughput against concurrency for every workload. The knee is workload-specific, which is why §6.2 reports that a pooled fit is meaningless.

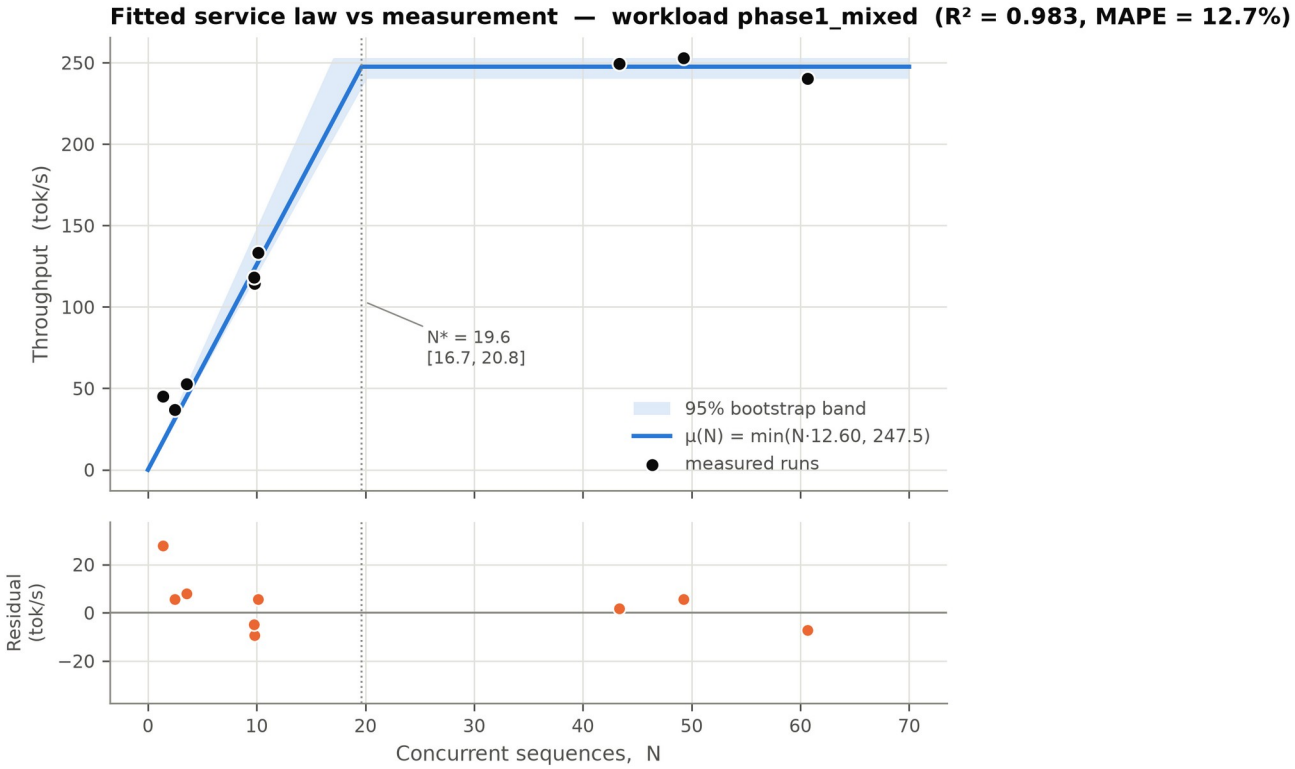

Figure 8. The fitted service law against its measurements for `phase1_mixed`, the only workload that both spans the knee and has a flat plateau.

---

6. Where the rate model breaks

This section is the point of the exercise, and it motivates §7.

6.1 μ_max is unidentified wherever the sweep did not saturate. `phase2_mixed` and `phase2_chat` have plateau slopes of +16.7 and +30.5 tok/s/seq — their "saturated" regions are still climbing steeply, because those sweeps reached only N ≈ 12–18. Their fitted μ_max and N* are artefacts of the N range. Experiment B was designed to fix this by sweeping to failure at N ≈ 117 and 129, and **it does not**: high-third slopes are 3.06 and 3.99 tok/s/seq, still rising. The reason is §7 — *the engine runs out of memory before it runs out of throughput.* On this device, under speculative decoding, μ_max is not merely unmeasured but unreachable.

6.2 r_0 is not workload-invariant, so the pooled fit is meaningless. Fitted r_0 spans 8.65–63.27 tok/s/seq — a factor of 7 — across workloads differing only in prompt/output length mix. Pooling gives $R^2 \approx 0.22$. The model is a *per-workload* law; any claim that pools workloads averages incompatible regimes.

6.3 The model has no memory dimension, so it cannot predict failure. μ(N) describes rate and says nothing about survival. It sees N = 256, predicts μ = μ_max, and has no term that fails. The remainder of this paper is about the term it is missing.

Additional limitations — Jensen bias from fitting on run-averaged N, non-stationarity from thermal throttling, the ambiguity of "service rate" under speculation, and the endogeneity of N under open-loop arrivals — are carried in `docs/queueing_model.md` §5.4–5.7 and summarised in §10.

7. The memory boundary

This section is ordered as the investigation ran: first the observation that the capacity signal does not work (§7.1), then the identification of what actually binds (§7.2), the closed form and its tests (§7.3–7.4), and a matched control that removes the mechanism and with it the failure (§7.5).

7.1 Occupancy does not explain the failures

The engine's reproduction behaviour is stochastic, and that turns out to be the most informative thing about it. Repeating one configuration as independent trials — fresh server, fresh engine, a load ramp escalating until the engine dies or the ramp is exhausted — the 3B boundary reproduced in **5 of 10** trials, a 50% rate with a Wilson 95% interval of [24%, 76%] (`docs/experiment_b.md` §8). A trial that survives is not one that failed to reach the boundary: it reaches the same concurrency ceiling and keeps serving.

The survivors invert the ordering that any occupancy-based account requires:

Arm	Peak KV when it died	Peak KV when it survived
`1.5b`	28.2%, 28.5%, 28.7%	—
`3b`	57.4%, 59.1%, 59.5%, 59.6%, 60.9%	80.7%, 80.8%, 80.9%, 81.3%, 82.6%

If occupancy were the binding constraint, no trial could survive at an occupancy above the lowest at which another trial died. That ordering is violated across the whole 3B set: **every** survivor peaked higher than **every** death, with a gap of nearly 20 percentage points between the highest death (60.9%) and the lowest survival (80.7%). The separation is not marginal and the two groups do not overlap.

The inversion is not an artefact of drift between batches. The first three trials ran back to back within nine minutes on one engine at one seed, and contain the inversion on their own: a death at 57.4%, a death at 59.6%, then a survival at 80.7%. The remaining seven trials, run about nine hours later, reproduce the same pattern and extend the survivor range to 82.6%.

Occupancy is therefore not merely a poor predictor of failure here; it is not monotonically related to it. **No threshold on this signal separates the runs that died from the runs that lived** — which is precisely what an admission controller keyed on occupancy would need. The argument requires no model and no theory of the mechanism: two runs of one configuration, one dead at low occupancy and one alive at high.

Figure 9 puts the same point geometrically, for the Experiment A campaign. A KV-bound system fails along the horizontal axis, occupancy rising to 100%; this one fails along the vertical axis, N reaching `max_num_seqs`, with occupancy at 28.4% ± 0.3%. The dashed line traces where occupancy would have to go for KV to be the binding resource, and the trials never approach it. The figure also marks the band that vLLM's KV-based admission control does guard — occupancy approaching exhaustion, where preemption and recompute engage — and the failures fall well outside it. **Figure 3** shows the same three trials against concurrency, dying with 71.6% of the cache unused.

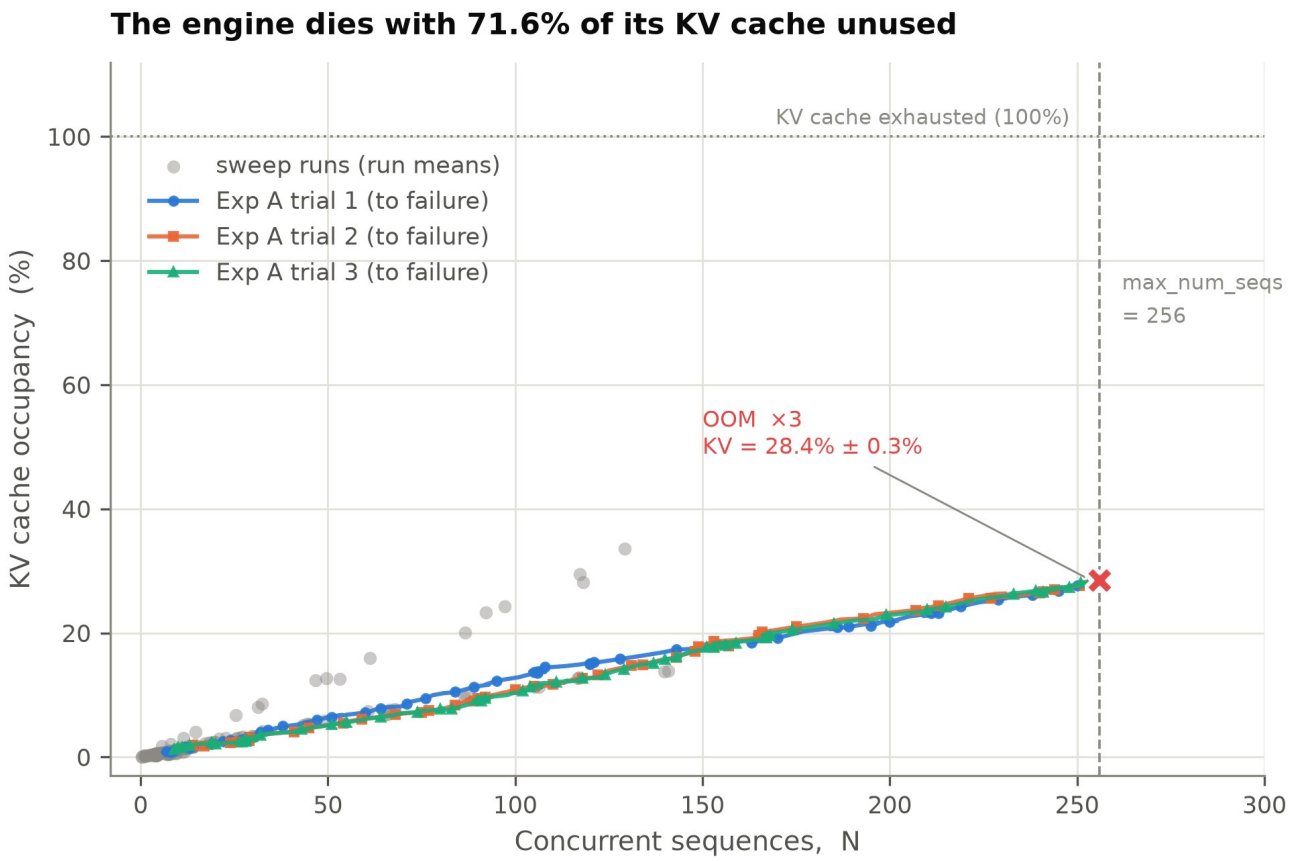

Figure 3. KV occupancy against concurrency for the Experiment A trials. All three die with 71.6% of the cache unused.

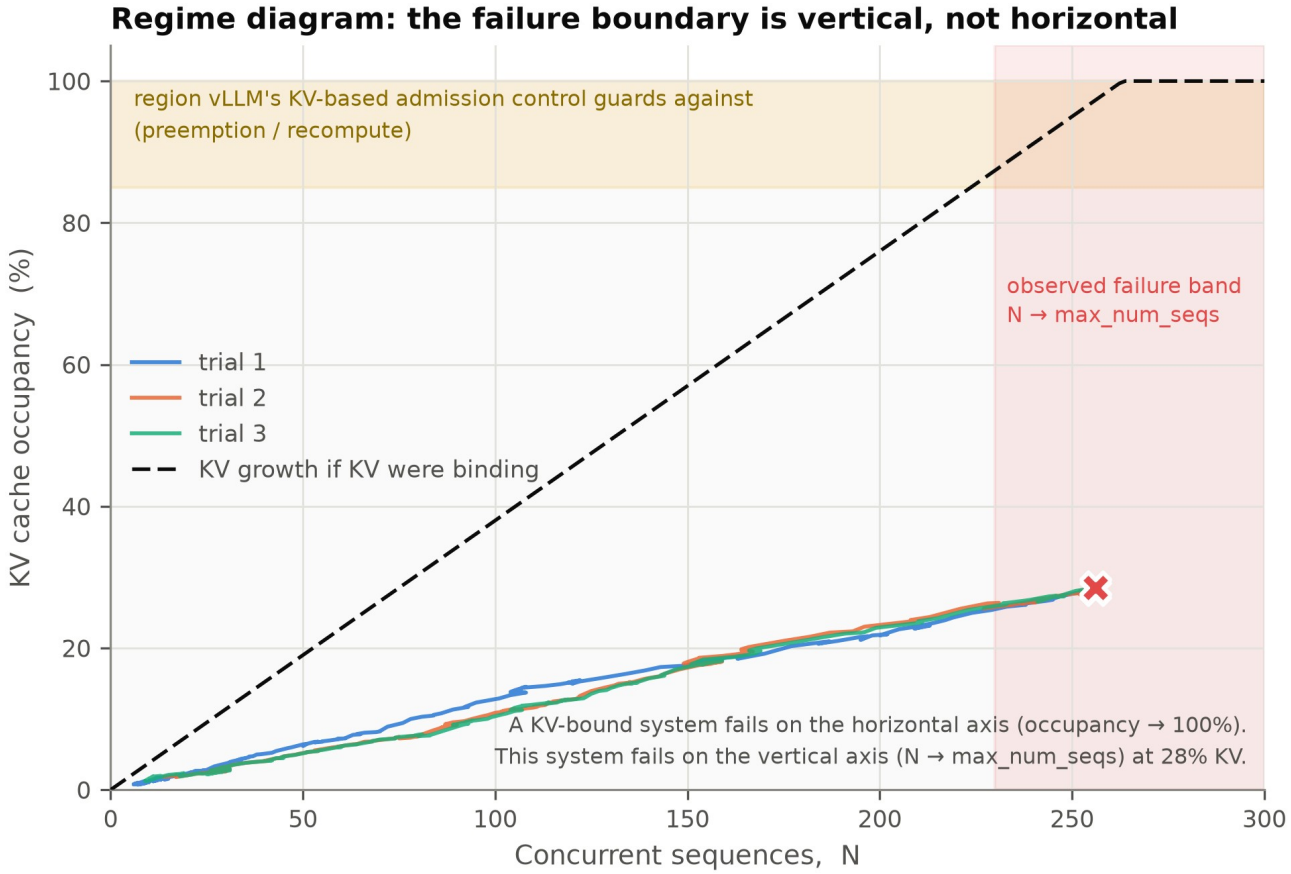

Figure 9. A KV-bound system fails along the horizontal axis, occupancy reaching 100%. This one fails along the vertical axis, N reaching `max_num_seqs`, at 28.4% ± 0.3% occupancy. The shaded band is the region KV-based admission control guards; the failures fall outside it.

A second, independent line of evidence comes from an experiment that varied speculative depth (§7.4). Its k = 7 arm died in 3/3 trials at N = 186, 187 and 256 against a sequence cap of 256, with occupancy at the failing step of **43.6%, 43.6% and 63.3%** against an estimated KV ceiling of N ≈ 363. Up to 37% of the pool was free at the moment of death, so neither the cap nor exhaustion of the pool accounts for it.

We state that arm on occupancy rather than concurrency deliberately: one of its three deaths lands on the cap itself, so a concurrency-only argument would not carry. The occupancy argument covers all three.

7.2 The allocation that does

Every OOM in this campaign — **16 independent engine deaths across two model sizes and three experiments** — failed at the same source line: `batch_expansion.py:227` in `_contract_batch`, reached via `spec_decode_worker.py:794` `score_proposals`. At that site the scorer materialises

```
226:  all_probs    = target_probs.new_zeros(*all_tokens.shape, self._vocab_size)
227:  all_logprobs = target_logprobs.new_full(size=all_probs.shape, ...)
```

two dense `[N, k+1, V]` fp32 tensors back to back, where `all_tokens.shape` is `(contracted_bs, k+1)`. This yields the law

$$M_{scorer} = N \cdot (k+1) \cdot V \cdot 4 \text{ bytes}$$

Every observed OOM reports line 227, so line 226 had already succeeded and the *second* allocation failed. Transient peak demand at this site is therefore **twice** the reported failed allocation. This does not affect the law as tested — `M_scorer` predicts the size of the tensor that failed, and our test statistic is MiB per sequence computed from that — but it does affect any prediction of the *concurrency* at which death occurs, which depends on total headroom. This is a source reading, not an experiment, and is reported as such.

Figure 5 is the accounting gap in one picture. On the left, what vLLM reserves at startup for the 1.5B configuration: 2.89 GiB of weights, 2.02 GiB of activation headroom, 8.15 GiB of KV cache. On the right, what the device actually holds at the moment of failure: the weights unchanged, 2.31 GiB of KV in use, **5.84 GiB of KV reserved and never touched**, and 1.86 GiB of activations and scorer. Into that state the scorer requests 742 MiB with 736 MiB free, and the engine dies — six megabytes short, beside six gigabytes of reserved cache it is not permitted to use. The reservation discipline of §2.1 is working exactly as designed throughout.

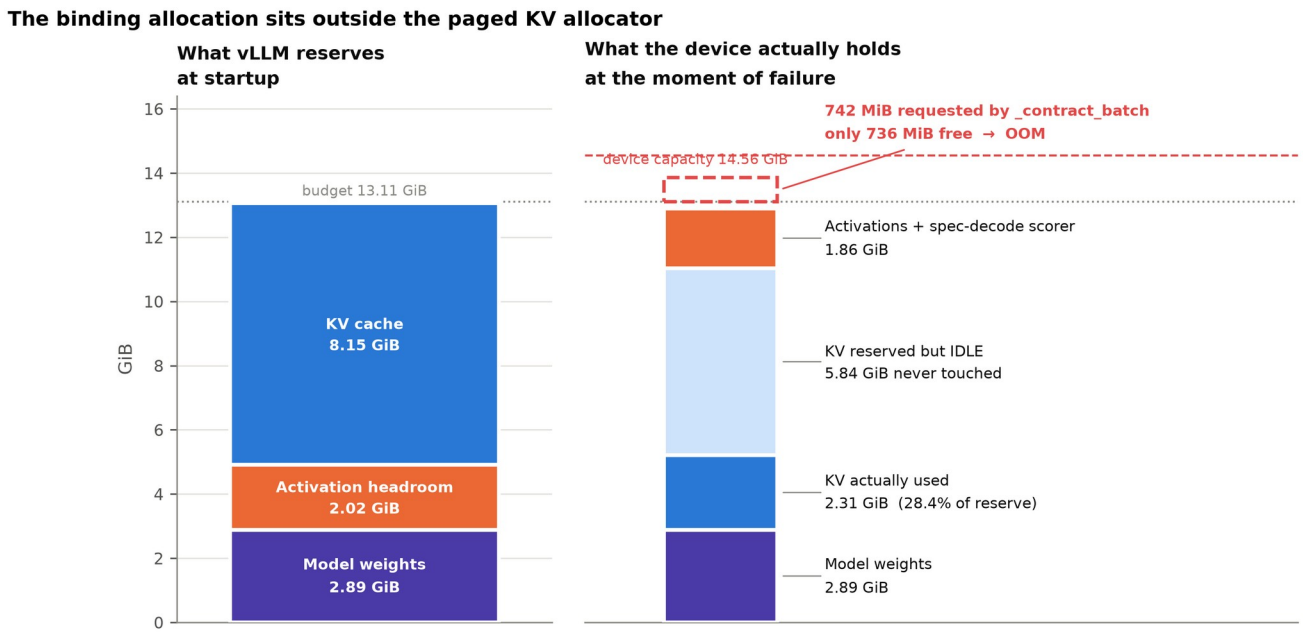

Figure 5. The accounting gap. Left: what vLLM reserves at startup. Right: what the device actually holds when it dies — 5.84 GiB of KV reserved and never touched, while a 742 MiB scorer request fails with 736 MiB free.

This allocation also explains §7.1's stochasticity, which an occupancy account cannot. The tensor is requested once per engine step as a single contiguous block; whether a request of that size succeeds depends on the state of the caching allocator at that instant — how fragmented the reserved-but-unallocated pool happens to be. Reported free memory at failure sits close to, but not below, the size of the allocation. Nothing about the boundary requires it to be crossed deterministically, which is why it must be reported as a rate rather than a threshold.

This last paragraph is a conjecture and we label it as one. The stochasticity is measured — 5 of 10 trials, with the Wilson interval above — but the allocator-state explanation for it is inferred from the shape of the data rather than instrumented. We did not record allocator fragmentation at the failing step, and a competing account (variation in the batch's context lengths, and so in free memory, at the moment the request is made) would fit the same observations. Distinguishing them needs allocator telemetry we do not collect. **The empirical claim stands without the explanation**: the boundary is crossed stochastically and must be reported as a rate.

7.3 The law on the N axis

With k+1 = 5 and V = 151,936, the law gives **2.898 MiB per sequence**. Against the ten deaths of Experiments A and B (`docs/experiment_b.md` §5):

Arm	N at failure	Predicted	Reported	Error
`1.5b` sweep	220	638 MiB	638 MiB	−0.1%
`3b` sweep	256	742 MiB	742 MiB	−0.0%
8 further probe trials	256	742 MiB	742 MiB	−0.0%

The 1.5B sweep died at lower concurrency and asked for correspondingly *less* memory, which is what makes this a test rather than a fit: the failing allocation tracks N, not model size. **Figure 4** shows the underlying trajectory —

device memory climbing with concurrency until the allocator has no room left for the next request.

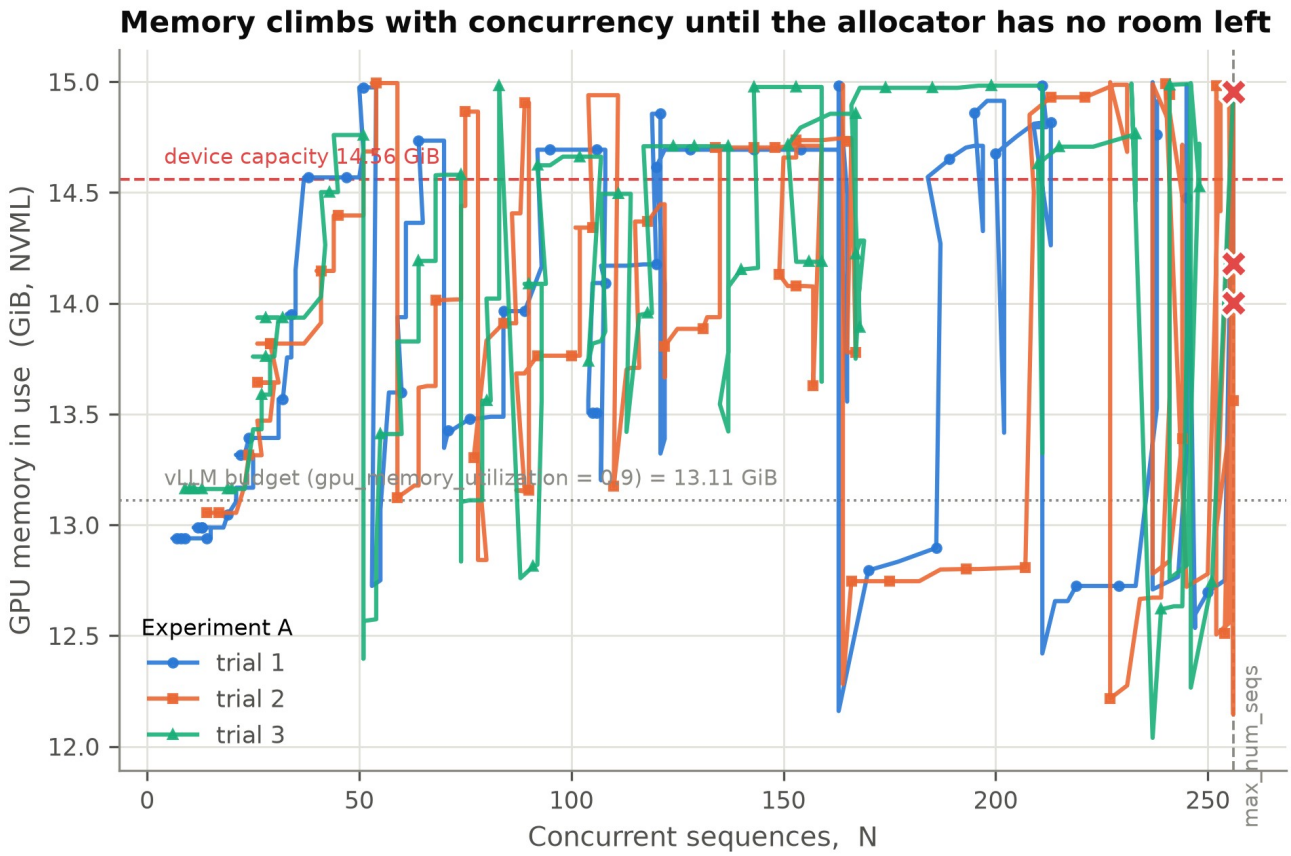

Figure 4. Device memory against concurrency, climbing until the allocator has no room for the next request.

7.4 The law on the k axis (Experiment F)

The N-axis test cannot identify the scorer specifically — any per-sequence allocation would look linear in N. The `(k+1)` factor is the discriminating term, and the one most likely to have taken another form, since how many speculative tokens are actually scored per step is a scheduler decision rather than simply the configured `k`. Experiment F varies it at fixed model, GPU, ramp and seed (`results/expF/expF_analysis.json`, **Figure 11**):

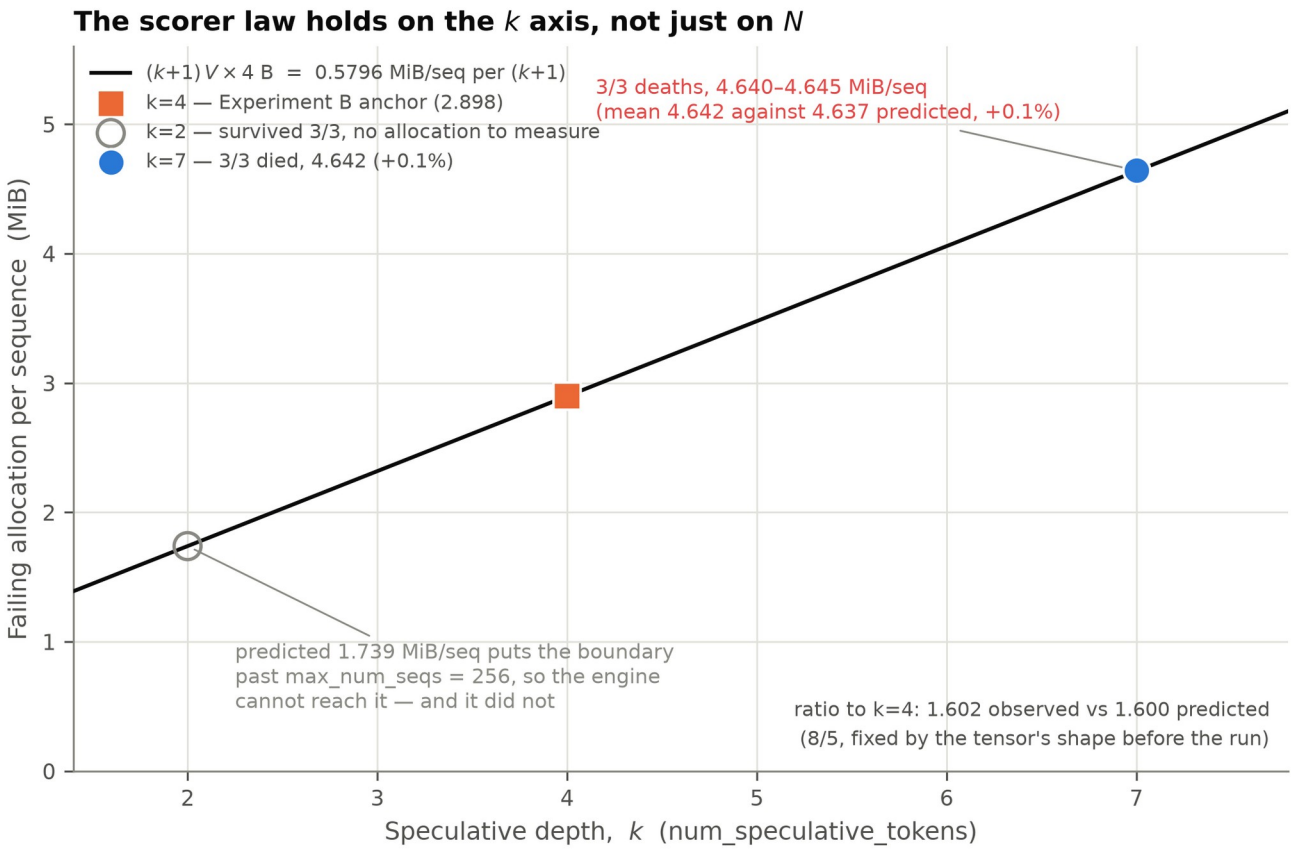

Figure 11. The failing allocation against speculative depth. k = 2 is drawn open because it survived: there is no failing allocation to measure.

Arm	k	Trials	Died	MiB/seq measured	Predicted	Error
Experiment B anchor	4	—	—	2.898	2.898	(anchor)
`k2_cap256`	2	3	0	— (no death to measure)	1.739	—
`k7_cap256`	7	3	3	4.642 (range 4.640–4.645)	4.637	**+0.1%**

The ratio to the k = 4 anchor is **1.602 observed against 1.600 predicted** — 8/5, fixed by the tensor's shape before the arm ran. The k = 2 arm is the cheap falsification the law had to survive: at 1.739 MiB/seq its boundary lies past the sequence cap, so the engine cannot reach it, and it did not — surviving 3/3 while reaching N = 256 and 83.1% occupancy. A law that over-predicted would have killed k = 2; one that under-predicted would have spared k = 7.

7.5 The mechanism control

Experiment F's fourth step re-ran the speculative-decoding contrast at `max_num_seqs` = 512, a cap chosen from a calibration sweep rather than guessed. Both arms share model, cap, ramp and seed; only `--speculative-config` differs.

Arm	Speculative decoding	Trials	Died	Peak N	Peak KV
spec on	on	3	3	410	98.96%

Arm	Speculative decoding	Trials	Died	Peak N	Peak KV
spec off	off	3	**0**	402	99.96%

The spec-on arm died 3/3 at **2.884 MiB/seq against 2.898 predicted (−0.5%)**, at allocations of 1167–1188 MiB — larger than anything in §7.3's table — and at N ≈ 410, extending the law's tested range from 220–256 out to 410.

The spec-off arm **died 0/3**, riding the ramp to λ = 8 at 99.91–99.96% occupancy — *higher* than the 98.66–98.96% at which the spec-on arm died — and kept serving. Remove the allocation the law names and the failure disappears, under a load that drives the same engine to a fuller KV pool than the one that killed it. The mechanism is not merely consistent with the deaths; it is necessary for them.

> **These particular deaths are not low-occupancy ones.** At cap 512 the spec-on arm reaches N ≈ 410 and a nearly full pool before the scorer allocation fails, so this pair does not demonstrate the low-KV OOM of §7.1 and §7.3. It demonstrates the *mechanism*, against a matched control. The two arms do different jobs: k = 7 shows the failure arriving with a third of the pool free; this pair shows it not arriving at all once the scorer is gone.

7.6 What moves with model size, and what does not

Fitting occupancy = a·N through the origin (bootstrap over runs, 2000 resamples): KV cost per sequence is 0.1103% for the 1.5B and 0.2501% for the 3B, a **ratio of 2.27× [2.17, 2.34]**. That ratio is predicted rather than merely observed, from two numbers neither of which came from the sweep: per-token KV cost rises 1.29× (36 layers against 28) and the pool it lands in shrinks 1.72× (fp16 weights of 5.79 GiB against 2.89 GiB leave 4.74 GiB against 8.15 GiB of KV). Together they predict **2.21×**, inside the measured interval; vLLM's own reported maximum-concurrency ratio (2.21×) agrees independently.

The scorer allocation, meanwhile, does not move with model size at all. **Figure 10** places the two panels side by side at the same scale, which is the whole result: KV cost per sequence moves with model size, and the allocation that kills the engine does not. **This is the bottleneck moving**: the wall sits at the same N for both models, but the 3B engine arrives there having spent twice as much of a smaller pool on KV. Extrapolating, the 3B arm would exhaust its pool at N ≈ 400, beyond the cap of 256 — which is why the scorer wall still arrives first. The margin is about 1.6× and closes from both ends at once. A modestly larger model on this device crosses over, at which point the failure changes identity and becomes a genuine KV exhaustion. **The claim is specific to a regime, and Experiment B locates its edge.**

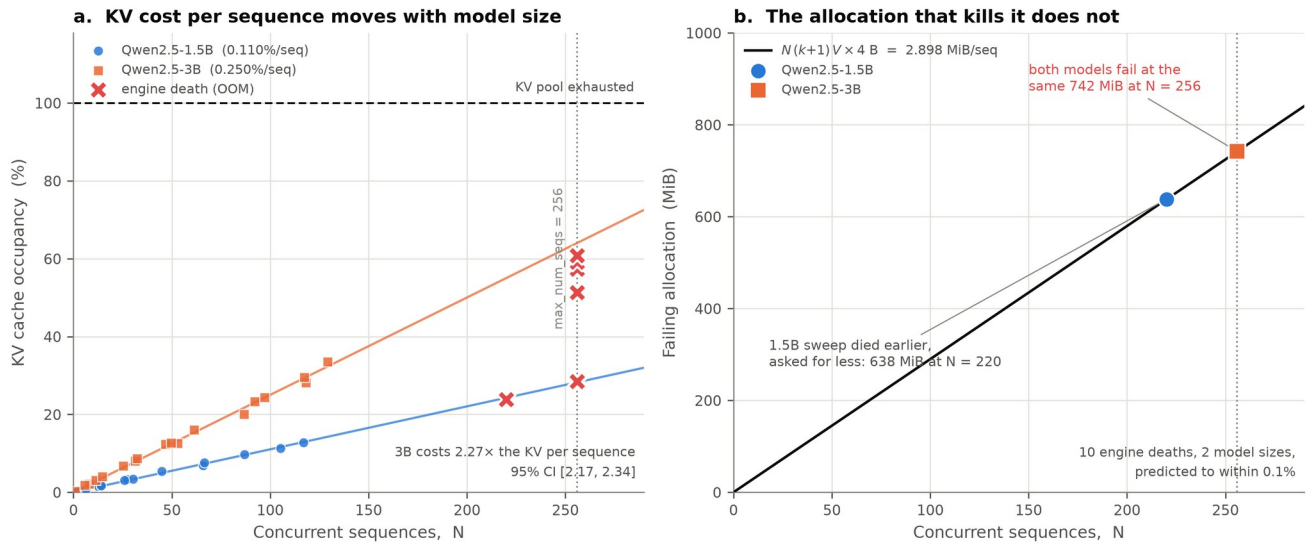

Figure 10. What moves with model size (KV cost per sequence, left) beside what does not (the allocation that kills the engine, right), drawn at the same scale.

Separately, speculative-decoding acceptance falls from 0.723 (1.5B, n = 17) to 0.589 (3B, n = 18), Hedges' g = −5.66, Welch p = 1.33e-17. The draft is prompt-lookup n-gram in both arms, so the proposal distribution is identical; the larger target model simply agrees less often. The cost is paid twice — fewer accepted tokens per step, and the same scorer tensor allocated to score them.

---

8. The regime flip: `max_num_seqs` is a memory parameter

The binding resource can be moved between the scorer and the KV pool by changing one scheduler parameter, on the same model, GPU and workload. vLLM sizes its memory-profiling pass at `max_num_seqs` and reserves the resulting activation peak, leaving the KV pool the remainder (`results/expF/calibration.json`):

`max_num_seqs`	Activation reserve	KV pool	Estimated KV-bound N	Regime
256	2.52 GiB	4.74 GiB	≈ 363	**scorer-bound** — dies at N = 256, KV 57–60%
512	2.89 GiB	4.38 GiB	≈ 336	scorer-bound
768	4.26 GiB	3.00 GiB	≈ 230	—
1024	5.64 GiB	1.63 GiB	≈ 125	**KV-bound** — survives, KV ≈ 99%, N ≈ 125

Raising the cap by 4× costs 3.1 GiB of activation reservation, taken directly out of the pool the same setting is presumed to govern. **Raising the admission limit therefore halves the concurrency the engine can actually sustain**, and flips which resource binds.

This has a methodological consequence we report because it cost us two experiments: an earlier attempt at the

speculative-decoding contrast used `max_num_seqs` = 1024 and measured the KV-bound row rather than the intended question. The cap in §7.5 was chosen from the calibration sweep above rather than guessed, for exactly this reason.

---

9. Admission control governs aggregate capacity, not only differentiation

Phase 2 compares five admission policies (`nocap`, `fcfs`, `srpt`, `edf`, `adaptive`) across workloads and arrival rates. Intervals are two-sided 95% t-intervals; effect sizes are Hedges' g [14] with the small-sample correction; contrasts use Welch's unequal-variance t [15], Holm- adjusted [13] across the **whole family of 207 comparisons** (`results/stats/STATISTICS.md`). Proportions carry Wilson score intervals [16], which do not degenerate at attainment near 0 or 1 as the normal approximation does — and several cells here sit at both extremes.

At n = 3 the comparison had almost no resolving power. Before Experiment D, with every cell at n = 3, exactly **1 of 171** contrasts survived Holm — and it was a queue-wait result, not a capacity one (`results/stats/STATISTICS.md` at commit `d9602c6`). Repeating the decisive cell at n = 10 (`phase2_mixed_n10`, λ = 4) changed that: of 207 comparisons, 32 are significant before correction and **9 survive Holm**. The surviving contrasts include:

Contrast	Metric	Effect	g	p_holm
`nocap` vs `fcfs`	SLO attainment	+0.888	+9.24	0.0000
`nocap` vs `fcfs`	Queue wait p99	−5.78e3 ms	−15.52	0.0000
`nocap` vs `fcfs`	**Throughput**	**+178 tok/s**	+2.67	0.0088
`edf` vs `fcfs`	SLO attainment	+0.412	+3.69	0.0000
`srpt` vs `fcfs`	SLO attainment	+0.397	+3.78	0.0000

Figure 6 shows aggregate SLO attainment by policy and arrival rate across the Phase-2 grid, and **Figure 7** breaks the same attainment down by workload class, where the differentiation these policies are designed for is visible: `nocap` holds every class above 0.9 in the n = 10 cell while `fcfs` collapses on all four.

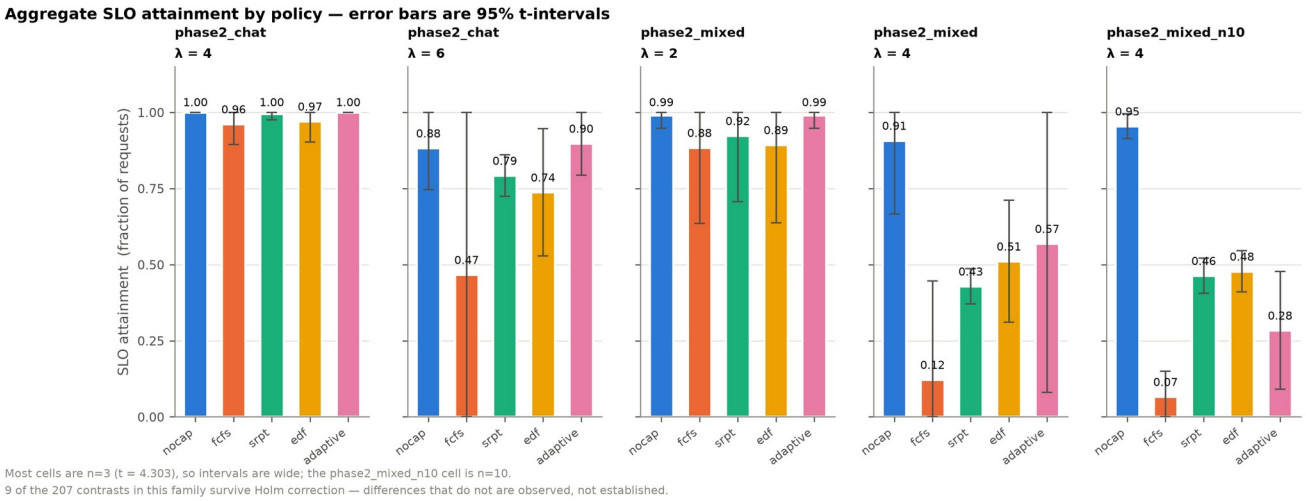

Figure 6. Aggregate SLO attainment by policy and arrival rate across the Phase-2 grid.

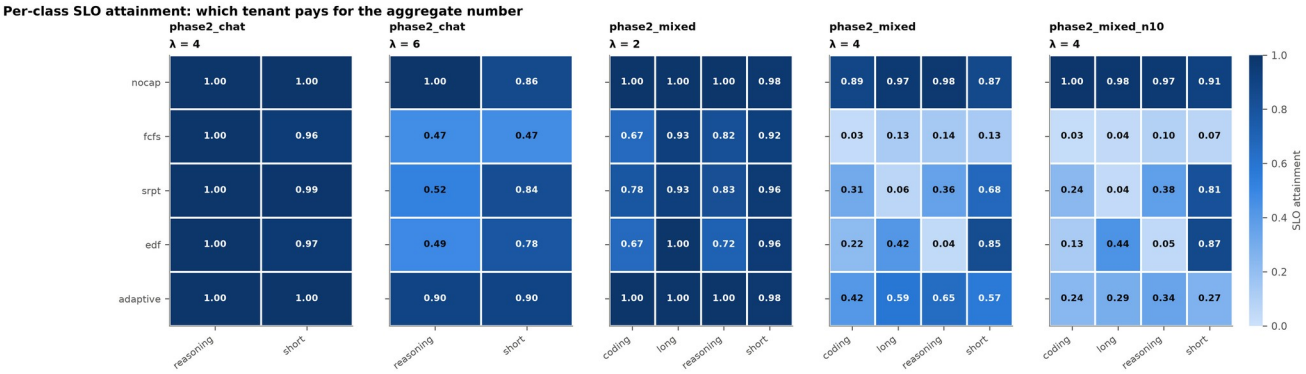

Figure 7. Per-class SLO attainment. In the n = 10 cell `nocap` holds every class above 0.9 while `fcfs` collapses on all four.

The throughput row is the one that matters for framing. Admission policy is commonly described as trading latency or fairness *at fixed capacity*. Here an uncapped queue gains **both** +0.888 SLO attainment **and** +178 tok/s (287 → 464) over FCFS, with both effects surviving the conservative correction. **Admission control governs aggregate capacity and differentiation together**, and the two would decouple only above the saturation knee — which §6.1 shows this hardware cannot reach before failing.

---

10. Threats to validity

10.1 Single device, single version, single model family. One passively cooled T4, vLLM 0.8.5.post1, and two Qwen2.5 checkpoints. This is the weakest axis of the paper and the first thing a referee will raise. Nothing here establishes that the constants generalise; the *mechanism* is a design property, but its magnitude on other hardware is unmeasured.

10.2 The failing code path does not exist in vLLM V1. Read from the installed source: `vllm/v1/spec_decode/` contains `eagle.py`, `ngram_proposer.py`, `metadata.py`, `metrics.py` and `utils.py` — and no `batch_expansion.py`. V1's equivalent, `vllm/v1/sample/rejection_sampler.py`, operates on a flattened 2-D

`(num_tokens, vocab_size)` tensor rather than a padded 3-D `[N, k+1, V]` expansion, and shows no sign of the doubled probs/logprobs pair. The honest framing is that **the specific failure is V0-specific, while the mechanism is a design property whose persistence in V1 is an open empirical question**: `num_tokens` in V1 is still ≈ N·(k+1) in the worst case, so the magnitude may well survive the refactor. Nothing here settles that, and only a V1 run would. This was a source read, not an experiment.

10.3 The V term of the law is asserted, not measured. Every death in the campaign ran a Qwen2.5 checkpoint at V = 151,936. The vocabulary dimension is read off the allocation site — `new_zeros(*all_tokens.shape, self._vocab_size)` — which is a direct reading of the code that fails, not an inference from the failure, but it is **not an independent measurement and must not be reported as one**. Consequently "the allocation is model-size independent" is demonstrated on two models that agree on precisely the parameter which would have made it size-*dependent*. Measuring V requires a model from a different family, which changes layer count, KV cost per token and activation profile simultaneously — so a disagreement could not be attributed to V. The arm that would settle it was scoped and **deliberately not run** (§14.4); this limitation is therefore permanent for this paper rather than pending.

10.4 Thermal throttling. This is a measurement of ours, not an appeal to a general phenomenon: NVML reported the throttle flag set, at 85–89 °C, through nearly the whole campaign (`thermal_throttled` is recorded per stage in every run record). The consequence is that absolute rates are depressed by an unknown factor and only paired and ratio comparisons are safe. Repeats within a cell ran back to back, so a thermal excursion is shared within a cell rather than averaged across it, which narrows intervals that should be wide.

10.5 Statistical scope. Holm correction assumes the family is the one reported. Adding metrics later without re-running the correction reintroduces the multiplicity it removes. Phase 2 outside the n = 10 cell remains at n = 3, where intervals are wide and effect sizes imprecise.

10.6 Model-fitting caveats. Jensen bias from fitting on run-averaged N systematically overestimates the service rate near the knee (μ is concave); N is endogenous under open-loop arrivals, so regressing X on N fits an equilibrium relation rather than a causal response; and "service rate" is ambiguous under speculation, where emitted tokens/s, steps/s and accepted tokens/s are three different quantities with three different ceilings.

10.7 A silent data-corruption artefact, and a corrected published number. When the engine dies mid-window, requests already streaming receive a **cleanly terminated** response — `success=True`, HTTP 200, no error — carrying roughly 5 tokens instead of the ~134 requested. Such runs report a plausible concurrency with a throughput an order of magnitude too low and are silently admitted by any filter that checks only the failed-request count. The output-length histogram spikes at multiples of `k+1`, because the stream ends on a speculative chunk boundary. Runs are now excluded on `actual_over_requested < 0.4`; the separation is bimodal rather than tuned, with every healthy run in the campaign in [0.75, 1.00] and every affected one in [0.02, 0.06]. **This affected a previously published number**: the same artefact is present in the frozen `paper-v1` campaign, in `modelval_mixed` λ=4 rep2, and was included in that fit. With it removed, that workload's fit improves from $R^2 = 0.558$ to 0.825 and its MAPE from 77.3% to 29.1%. Any reader comparing against `paper-v1` should use the corrected figure.

---

11. Claim status table

Because the distinction is easy to lose in prose, every substantive claim is labelled:

Claim	Status
KV occupancy is non-monotonically related to failure	**Empirical** — every 3B survivor peaked above every 3B death (§7.1)
Arrivals Poisson; service requirement generally distributed	**By construction**
$N \le$ `max_num_seqs`	**Architectural**
$\mu(N)$ non-decreasing and concave	**Theoretical** — batching amortises a fixed per-step cost
$\mu(N) = \min(N \cdot r_0, \mu\_max)$	**Modelling assumption** — $R^2 = 0.983$ on `phase1_mixed`, ≈ 0.22 pooled
$r_0 = 12.60$, $\mu\_max = 247.5$, $N^* = 19.7$	**Empirically fitted** (`phase1_mixed`)
~104 decode steps/s bandwidth ceiling	**Derived from specs, and not attained**
The engine OOMs in the scorer at low KV occupancy	**Empirical** — 16 deaths, and outside the rate model
`M = N·(k+1)·V·4` predicts the failing allocation	**Empirical on N and (k+1)**; V asserted (§10.3)
Peak demand at the site is 2× the reported allocation	**Source reading**, not measured
Removing speculation removes the failure	**Empirical** — 0/3 vs 3/3 at matched cap and higher occupancy
Raising `max_num_seqs` flips the binding resource	**Empirical** — calibration sweep + both regimes observed
Admission control governs aggregate capacity	**Empirical** — n = 10, survives Holm over 207 comparisons
The mechanism persists in vLLM V1	**Open** — source suggests the path is gone; unmeasured

---

12. Reproducibility

Two frozen snapshots (`snapshots/paper-v1`, `snapshots/paper-v2`) are physical copies, not git tags, because later sweeps regenerate summaries and figures in place. Each carries code, results, figures, gzipped driver logs, a full `ENVIRONMENT.json` and a `MANIFEST.sha256` over every file. Where the two disagree, `paper-v2` is the corrected one (§10.7).

```
python tools/fit_queueing_model.py --results-dir results   # service-rate fits
```

```
python tools/analyze_expB.py                                  # experiments A, B and F ->
docs/experiment_b.md
python tools/analyze_expF.py                                  # experiment F arms
python tools/analyze_stats.py                                 # phase-2 policy statistics
python tools/make_paper_figures.py                            # figure set, including
Figure 11
```

`docs/experiment_b.md` is generated, not hand-edited: it regenerates byte-identically from the committed results, and degrades cleanly to its pre-F content if `results/expF` is absent.

---

13. Conclusion

We set out to fit a service-rate model to a continuously-batched LLM server and found that the model could not be identified, because the engine died before it saturated. That failure turned out to be the result. On the system we measured, the engine never once ran out of the resource its capacity signal reports.

The evidence for that is a pair of runs rather than an argument. Under one configuration, at one seed, minutes apart, the engine died with the KV pool 57.4% full and then survived a full load ramp at 80.7%; across ten trials every survivor peaked higher than every death, with no overlap between the groups. Nothing in an occupancy threshold can separate those runs, which is what an admission controller keyed on occupancy is asked to do. What binds instead is an allocation in the speculative-decoding scorer, `N·(k+1)·V·4` bytes, sitting outside the paged allocator and absent from every gauge the engine exposes; it predicts the failing allocation to within 0.1% at two speculative depths and to within 0.5% at a concurrency 60% beyond anything else we measured, and removing it removes the failure at a *higher* occupancy than the one that killed the speculative arm.

Two consequences seem to us more durable than the constants.

The reservation is only as wide as the allocator it names. Orca's admission-time guarantee is sound, and our engine dies with it fully intact — six megabytes short of a scorer allocation while six gigabytes of reserved KV sit untouched (Figure 5). A memory-safety argument that ranges over one allocator is not a memory-safety argument for the system, and continuous batching has accumulated allocators faster than it has accumulated accounting for them.

A capacity limit that is also a memory parameter can invert. `max_num_seqs` bounds concurrency and sizes the activation reserve deducted from the KV pool, so raising it from 256 to 1024 does not raise the sustainable concurrency but roughly halves it, and moves the binding resource from the scorer to the pool. Orca kept these as two knobs; collapsing them into one gives the operator a control whose sign changes partway along its range. That is a property of an implementation, not of a queueing discipline, and it is invisible to the theory that would otherwise describe the system.

None of this says speculative decoding is a poor design, or that we have found a defect. The allocation does exactly what it was written to do. The finding is that its size is unbounded in a dimension nothing admits against — and that the discipline which makes serving systems safe, the reserve-before-admit rule, is stated over an allocator rather than over a device.

We measured one GPU, one engine version and one model family, and §10 is explicit that the constants should not be carried elsewhere. The question we think worth carrying is smaller and harder to dismiss: for any serving system, what does its capacity signal *not* cover, and has anyone checked whether the uncovered part is the part that binds?

---

14. Open items before submission

Blocking:

1. ~~**Related work and bibliography do not exist** (§2).~~ **Done.** §2 is written and §15 lists 16 references, all cited and none orphaned. Eight were read in full and their cited passages transcribed from the papers; three are metadata-only (**[M]**) and cited only where an [R] source corroborates the claim. The remaining risk is not missing citations but *unread* ones: [8], [9] and [12] should be obtained before submission, and any reviewer-facing claim about [11] should be checked against its qualification, since that paper exists to complicate the consensus it records.
2. ~~**Narrow the abstract** to the scope §10 supports.~~ **Done.** The abstract now names the machine, engine version and model family in its first sentence, states that the `V` term is read rather than measured, notes that the failing path is absent from vLLM V1, flags the thermal caveat on absolute rates, and closes on a scope paragraph that declines the general claim. It should be re-checked against §10 after any further result lands — the failure mode to watch for is the scope paragraph drifting out of step with §10.1–10.4.
3. **Choose a venue and format.** No LaTeX toolchain is installed on this machine. Prior assessment of realistic venues: MLSys (competitive), IEEE TPDS, ACM TOMPECS, Performance Evaluation, JPDC, FGCS, Cluster Computing, IEEE Access; a workshop such as EuroMLSys or HotInfra first is reasonable. Not OSDI/SOSP/NSDI/ASPLOS — single GPU, no system built, narrow mechanism.

Closed by decision — not pending, not queued:

4. **The different-vocabulary arm will not be run.** It would close §10.3, the one untested term of the law, by measuring a model with a different `V`. The arm was scoped: `meta-llama/Llama-3.2-1B-Instruct`, V = 128,256, predicting 2.446 MiB/seq and 656.7 MiB at N = 256 against Qwen's 741.9 — a 12.5% separation the campaign's ±0.5% agreement would resolve easily. It is not blocked by anything technical: the model is gated, and accepting the licence plus an `HF_TOKEN` would unblock it. **The decision is to submit on the current experiments instead**, taken 2026-08-11.

The consequence is carried openly rather than left implicit. `V` remains **read from the allocation site, not measured** (§10.3), the law is described throughout as tested on two of its three axes, and "the allocation is model-size independent" rests on two checkpoints that share the one parameter which would have made it size-*dependent*. A referee will raise this; the abstract, §7.3, §10.3 and §11 all state it before they can. What must not happen is the claim quietly widening — if any future edit describes the law as fully tested, this item is the reason that is wrong.

For the record, should the decision be revisited: Llama-3.2-1B is the right choice because the V test needs a model that *reaches* high concurrency. The cached larger-vocabulary alternative (Phi-4-mini, V = 200,064) costs 128 KiB/token, plateaus near N ≈ 61, goes KV-bound and never reaches the scorer at all, so it cannot test the law however convenient it is to run.

Would most raise the ceiling, if the scope is ever widened:

5. **A second GPU class** kills the T4-specific objection (§10.1).
6. **A current vLLM version** kills the stale-bug objection and answers §10.2 directly.

Non-blocking discrepancy noted while assembling this draft: `docs/queueing_model.md` reports the pooled R^2 as **0.229** in its §4 table and **0.216** in its §5.2 text. Both round to ≈ 0.22 and neither changes any conclusion, but one of them is stale and they should be reconciled before either is quoted.

---

15. References

Read status is recorded per entry, because the two are not the same thing and the distinction governs what each may be cited for. **[R]** = full text obtained and the passages cited here read directly; every quotation and section pointer in §2 attributed to an [R] entry was taken from the paper itself. **[M]** = metadata and headline contribution verified against a primary or near-primary source, full text not obtained; an [M] entry must not be cited for a quotation or a fine-grained claim. All author lists were taken from the papers themselves, except [8] and [9], whose metadata comes from the publishers' records. Checked August 2026; sources listed in `docs/related_work.md` §6.*

1. **[R]** Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. "Orca: A Distributed Serving System for Transformer-Based Generative Models." *OSDI 2022*, 16th USENIX Symposium on Operating Systems Design and Implementation, Carlsbad CA, 11–13 July 2022, pp. 521–538. ISBN 978-1-939133-28-1.

2. **[R]** Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., Stoica, I. "Efficient Memory Management for Large Language Model Serving with PagedAttention." *SOSP 2023*, ACM SIGOPS 29th Symposium on Operating Systems Principles. DOI 10.1145/3600006.3613165. arXiv:2309.06180.

3. **[R]** Leviathan, Y., Kalman, M., Matias, Y. "Fast Inference from Transformers via Speculative Decoding." *ICML 2023*, PMLR 202:19274–19286. arXiv:2211.17192.

4. **[R]** Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., Jumper, J. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318,

5. *Preprint — check for a peer-reviewed version before submission.*

6. **[R]** Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., Ramjee, R. "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve." *OSDI 2024*. arXiv:2403.02310.

7. **[R]** Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., Lin, W. "Llumnix: Dynamic Scheduling for Large Language Model Serving." *OSDI 2024*. arXiv:2406.03243.

8. **[R]** Patke, A., Reddy, D., Jha, S., Qiu, H., Pinto, C., Narayanaswami, C., Kalbarczyk, Z., Iyer, R. "Queue Management for SLO-Oriented Large Language Model Serving." *SoCC 2024*, ACM Symposium on Cloud Computing, Redmond WA, 20–22 Nov 2024. DOI 10.1145/3698038.3698523. arXiv:2407.00047.

9. **[M]** Zhang, J., Zwart, B. "Steady State Approximations of Limited Processor Sharing Queues in Heavy Traffic." *Queueing Systems* 60:227–246, 2008. *Paywalled; not obtained. Cited only for the existence of the heavy-traffic steady-state result, which [10, §1] corroborates.*

10. **[M]** Zhang, J., Dai, J. G., Zwart, B. "Law of Large Number Limits of Limited Processor-Sharing Queues." *Mathematics of Operations Research*, 2009. DOI 10.1287/moor.1090.0412. *Paywalled; not obtained. [10, §1] describes it as the fluid approximation this line of work builds on, and it is cited on that basis.*

11. **[R]** Zhang, J., Dai, J. G., Zwart, B. "Diffusion Limits of Limited Processor Sharing Queues." *Annals of Applied Probability* 21(2):745–799, 2011. DOI 10.1214/10-AAP709. *Open access; §1 read directly. All LPS quotations in §2.4 are from this paper.*

12. **[R]** Sadhukhan, R., Chen, J., Chen, Z., Tiwari, V., Lai, R., Shi, J., Yen, I. E.-H., May, A., Chen, T., Chen, B. "MagicDec: Breaking the Latency-Throughput Tradeoff for Long Context Generation with Speculative Decoding." *ICLR 2025*. OpenReview `CS2JWaziYr`. *Cited in §2.2 for the conventional wisdom it records and for the qualification it adds; note that the paper's purpose is to show the conventional wisdom does not hold in the long-context regime, so it must not be cited as endorsing the reversal unconditionally.*

13. **[M]** Wu, B., Zhong, Y., Zhang, Z., Liu, S., Liu, F., Sun, Y., Huang, G., Liu, X., Jin, X. "FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference." *NSDI 2026*, 23rd USENIX Symposium on Networked Systems Design and Implementation. Earlier preprint: "Fast Distributed Inference Serving for Large Language Models", arXiv:2305.05920. *Metadata verified from the USENIX programme and arXiv; full text not obtained. Cited only for the mechanism named in its title.*

Statistical methods. Standard references, cited for the procedures used in §7.1, §9 and §10.5.

13. Holm, S. "A Simple Sequentially Rejective Multiple Test Procedure." *Scandinavian Journal of Statistics* 6(2):65–70, 1979.

14. Hedges, L. V. "Distribution Theory for Glass's Estimator of Effect Size and Related Estimators." *Journal of Educational Statistics* 6(2):107–128, 1981. DOI 10.3102/10769986006002107.

15. Welch, B. L. "The Generalization of 'Student's' Problem when Several Different Population Variances are Involved." *Biometrika* 34(1–2):28–35, 1947.

16. Wilson, E. B. "Probable Inference, the Law of Succession, and Statistical Inference." *Journal of the American Statistical Association* 22(158):209–212, 1927.

No `[CITE]` gaps remain. Of the five originally open: the batch-size reversal is now [11], FastServe is [12], and the statistical procedures are [13]–[16]. Two were closed without a citation, deliberately. §10.4's thermal claim is a measurement of ours, recorded per stage in the run records, and is stated as such rather than leaning on a general result. §7.2's allocator-fragmentation account could not be supported and is now **labelled a conjecture in place**, with the competing explanation named and the empirical claim separated from it.